Tutorial 2

Aakash Aadhithya(DA24B028)
Course: Computer Systems(DA2304)

Part 1

(a)

The parser module reads the assembly file line by line and strips away all whitespace,blank lines, and comments and it breakdowns each valid instruction into the components like dest,comp,jump etc.
The code module is like a dictionary,it takes the string components produced by parser(D+M or JEQ) and converts them into binary codes as defined
The SymbolTable module is a memory table , it keeps track of all predefined symbols,label symbols and variable symbols.it maps these symbols to RAM or ROM addresses.

(b)

In First pass ,the assembler reads through the code strictly to find lablels in parrenthesis.it keeps a running counter of ROM(instruction memory) address.whenever it encounters a label,it adds that label to the symbol table along with the very next ROM address and no binary translation happens in this step.
In Second pass , The assembler reads the code from the top again.This time,it translates C-instructions into binary.when it encounters A - instruction(@X):

- If X is a number,it translates it directly to binary
- If X is a symbol already in the symbol table,it replaces it with the stored address
- If X is a new symbol not found in the table , it treats it as a new var, assigns it to the next available RAM adress(Starting at RAM[16]), addsit to the table and translates it.

Part 2:

(a)

Every memory address hoolds exactly one 16 bit integer and we are working with 3x3 matrices,each matrix contains exactly 9 elements. therefore,the memory requirement is 9 registers or 18 bytes per matrix and for all 3 matrices combines total of 27 registers is needed or 54 bytes.

- Matrix A : RAM[16] to RAM[24]

- Matrix B : RAM[25] to RAM[33]
- Matrix C : RAM[34] to RAM[42]

(b)

Hack ALU supports fundamental operations like addition ,subtraction ,bitwise AND,bitwise OR,and bitwise negation hbut it does not have multipliers.
To caluculate the product of two numbers ,the design choice is to use a repeated addition assuming both numbers are integers.
in code i used a localized loop called MULT_LOOP that does the following:

- A register (R3) is set up to keep a running total of the sum.
- the value from matrix A called multiplicand is stored in temporary register R9
- the value from matrix B called multiplier or counter is stored in other temporary register R10.
- Now execution happens , inside the loop ,program adds the value of R9 to the running sum in R3.it then decrements the counter in R10 by 1,this loop repeats units the counter hits zero(JLE)
- We can optimize this MULT_loop by iterating over a smaller number; i.e., (1000x2 is much better than 2x1000)

(c)

Static initializaton:
static initialization means hardcodng the matrix values directly into the ROM.when program starts,it exeeutes a seuence of instruction to write these predefined values into the data memory(RAM).
In hack assembly,we cannot write a value directly to a memory addess in one step.first you must load the value into the data register ($D\bar{A}$),point to target memory address(@address),and then store the value ($M\bar{D}$).
Dynamic initialization:
it means program is compiled one,and the matrix values are provided by the user while the program is running.
the simplest method is for the user to enter values into ram 16 - 33 (2 matrices so 18 inputs) using the CPU EMmulater interface before pressing run. then the assembly code would skip the setup instructions entirely and jump straight into mult logic.
another way is called Keyboard input:it is required reading from the hack computers memory mapped keyboard(KBD a RAM 24576).the program would run an input loop,waiting for the user to press keys.it would need to read the ASCII values of the keystrokes,convert them into integer values(handling multidigit numbers), and sequentially store them into the matrix pointers(A_PTR and B_PTR) until all 18 elements are filled.

Part 3:

Yes,the choice of memory layout affects the efficiency of the assembly program because of how matrix multiplication is calculated.caluclating a single cell in the resulting matrix C requires taking the dot product of a row from matrix A and a column from matrix B. and hack computer relies on 1D RAM arrays,we must use pointers to traverse these matrices. By storing matrix A in row-major order,the elemnents of each row sit in contigous memory addresses.by storing matrix B in column major order the elements of each colum also sit in contigous memory addresses.
Now lets see why effieciency is gained:Because the data we need next is always right next register in RAM,our inner loop can simply advance both pointer by 1 to fetch the next numbers to multiply.this takes exactly two lines of assembly code per pointer.
if we had storedd matrix B in row mahor order as well,its column elements would be separated by a gap of 3 RAM addresses.to find the next number in the column,the program would have to dynamically add 3 to pointer on every sing;e pass which consumes significantlu more instruction and wastes CPU clock cycles so we conclude that by aligning the data in RAM with the exact mathe matical traversal pattern we need,the program minimzes pointer updates and runs at maximum efficiency